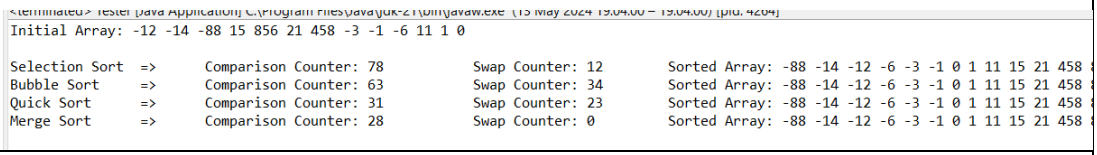
CSE222 / BİL505
Data Structures and Algorithms
Homework #6 – Report

Furkan Filicioğlu

1) Selection Sort

Time Analysis	In the Selection Sort, we have two for loop inside each other.First loop and second loops are the iterates as the length of the array.Complexity is o(n^2). Complexity is not efficient. Also,It makes unnecessarly comparisons even if the rest of the array is already sorted.There is no alghorithm to prevent it. For the small data sets,maybe it can be usefull.implementation is easy,but not useful.
Space Analysis	Since we were working on an existing array, we did not allocate a space in the heap again.Also we did not create any object or array inside the method.we can create int values or other variables but is not created in heap.Complexty is o(1). Space complexity is perfect but time complexty is worse.

2) Bubble Sort

Time Analysis	In the bubble Sort, we have two for loop inside each other.First loop is for the length of the array,for the second loop,maybe we are not looking for the size of array but we should think the worst case.So complexity is o(n^2).In this case,it is not usefull.On the other hand, when we are working on a sorted or partially sorted array,Bubble sort is much better than others.We does not compare whole elements.
	`Initial Array: 1 2 3 4 5 6 7 8`
	
	`Selection Sort => Comparison Counter: 28 Swap Counter: 7 Sorted Array: 1 2 3 4 5 6 7 8`
	`Bubble Sort => Comparison Counter: 7 Swap Counter: 0 Sorted Array: 1 2 3 4 5 6 7 8`
	`Quick Sort => Comparison Counter: 28 Swap Counter: 35 Sorted Array: 1 2 3 4 5 6 7 8`
	`Merge Sort => Comparison Counter: 12 Swap Counter: 0 Sorted Array: 1 2 3 4 5 6 7 8`
	
	Also in the big data sets,better than selection sort because make less comparision
	 
	`Initial Array: -12 -14 -88 15 856 21 458 -3 -1 -6 11 1 0`
	
	`Selection Sort => Comparison Counter: 78 Swap Counter: 12 Sorted Array: -88 -14 -12 -6 -3 -1 0 1 11 15 21 458`
	`Bubble Sort => Comparison Counter: 63 Swap Counter: 34 Sorted Array: -88 -14 -12 -6 -3 -1 0 1 11 15 21 458`
	`Quick Sort => Comparison Counter: 31 Swap Counter: 23 Sorted Array: -88 -14 -12 -6 -3 -1 0 1 11 15 21 458`
	`Merge Sort => Comparison Counter: 28 Swap Counter: 0 Sorted Array: -88 -14 -12 -6 -3 -1 0 1 11 15 21 458`

Space Analysis	Since we were working on an existing array, we did not allocate a space in the heap again.Also we did not create any object or array inside the method.So, Complexity is o(1).

3) Quick Sort

Time Analysis	İt depens on how to choice pivot correctly.if we choice pivot worst,this will be worst case which is O(n^2).In the best case and average case,complexty is o(nlogn) because of the for loop in the recursive calls.Because,we are dividing by two untill it sorted.for the best and average cases,it is much usefull when we compare to Bubble sort and Selection Sort.for the even100 input,Complexity will be very usefull
Space Analysis	Quick sort has space complexity of O(logn).Here,we are not creating any new array.We are making on the process in the same array but at each recursive cal,l a new stack frame of constant size must be allocated. Hence the O(log(n)) space complexity.

4) Merge Sort

Time Analysis	Merge sort is the one of the fast sorting algorithms. Complexty is different than Quick sort a little bit because, for the every cases(best,average,worst) Complext is o(nlogn).we don't have to think worst case so it is much useful.
	`Initial Array: 11 10 9 8 7 6 5 4 3 2 1`
	`Selection Sort => Comparison Counter: 55 Swap Counter: 10 Sorted Array: 1 2 3 4 5 6 7 8 9 10 11`
	`Bubble Sort => Comparison Counter: 55 Swap Counter: 55 Sorted Array: 1 2 3 4 5 6 7 8 9 10 11`
	`Quick Sort => Comparison Counter: 55 Swap Counter: 35 Sorted Array: 1 2 3 4 5 6 7 8 9 10 11`
	`Merge Sort => Comparison Counter: 17 Swap Counter: 0 Sorted Array: 1 2 3 4 5 6 7 8 9 10 11`
	Even if for the worst cases,it works better.

Space Analysis	Space complext is O(n) because creating two array and the total sizes of the arrays are equal to array' size.

General Comparison of the Algorithms

1)Bubble sort ande Selection sort is better than Quick and Quick is better than merge in terms of Space Complexity.

2)Merge Sort time complexity almost the same with the Quick sort but these are different in terms of worst case.For the worst case,Merge sort is more usable but its space complexity is worse than Quick Sort.

3)As ı said,Merge Sort and Quick Sort is better than Bubble Sort and Quick Sort in terms of time complexity.If we compare Bubble Sort and Quick Sort,space complexities are equal also time complexities are equal but if array is sorted,comparision will be less than Quick sort.So it is more usable

4)In conclusion, If the memory restricted, bubble and selection may be preferred, because of the use of less memory.But if time complexity is needed,sometimes Quick Sort sometimes merge sort more useful.